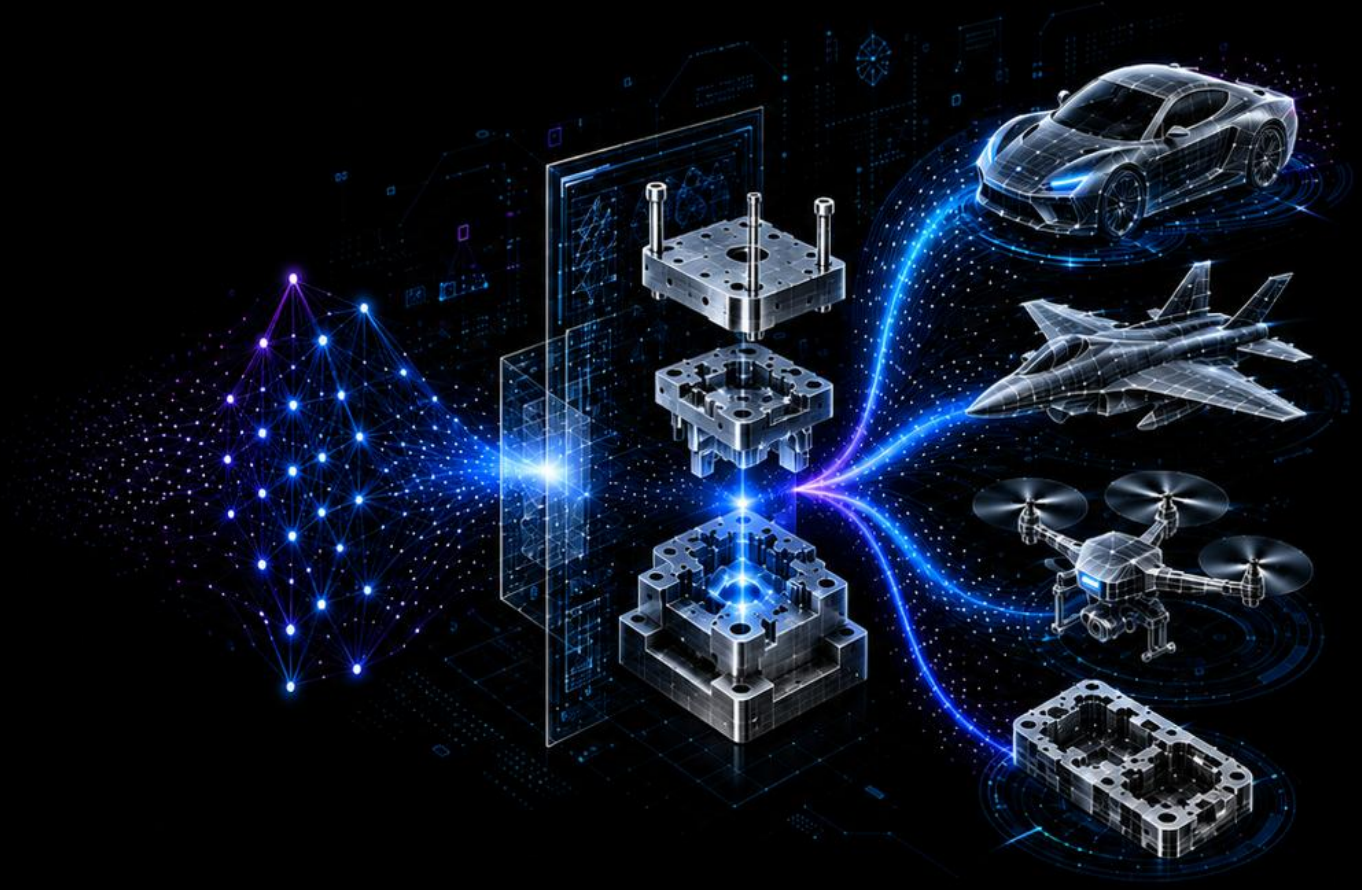
5장 CNN 합성곱 스터디

스터디 보고



정승한
(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

Convolution Neural Network

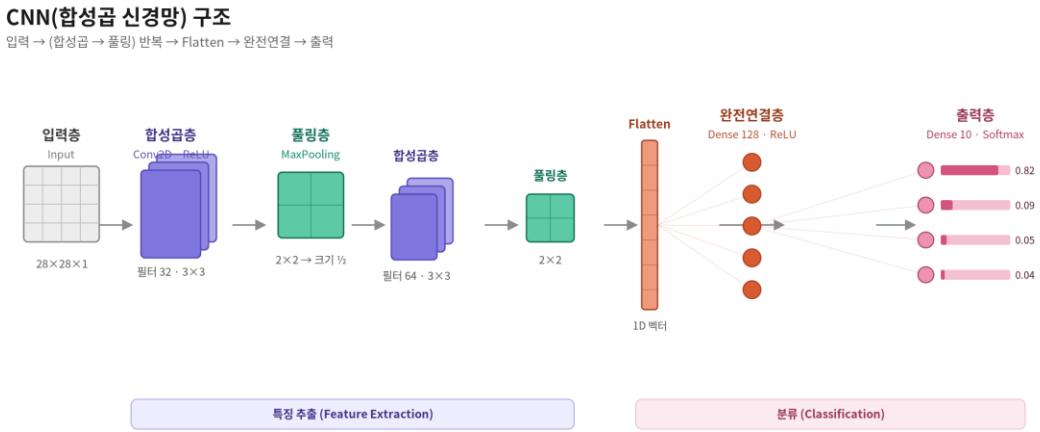
CNN

이미지 데이터는 방향, 채널 등의 공간적 구조를 가진 데이터이며, CNN은 이러한 공간적 패턴을 잘 잡아내는 구조를 가짐

구조

CNN은 크게 5개 종류의 층을 가짐

1. Input layer
2. Convolution layer
3. Pooling layer
4. Fully connected layer
5. Output layer



세부 설명

– Input layer

이미지를 숫자 배열로 받아들이는 층

높이, 너비, 채널 수로 행렬 형성

– Convolution layer

Kernel이라는 작은 창을 이미지 위에서 이동시키며 각 위치마다 곱한 값을 더해 feature map을 만든다

– Pooling layer

feature map의 크기를 줄여 계산량을 낮춤

구역을 나누고 구역 내 최댓값을 선택하는 Max pooling이 흔히 사용됨

– Fully connected layer

2차원 feature map을 1차원으로 펼친 후 모든 뉴런을 연결하여 최종 판단

– Output layer

뉴런 개수 = 분류할 클래스 개수이고 활성화 함수로 확률을 내어 분류

Softmax는 다중분류, sigmoid는 이진분류

Convolution Neural Network

Convolutional Layer

입력 데이터에서 특성을 추출하는 역할

Kernel

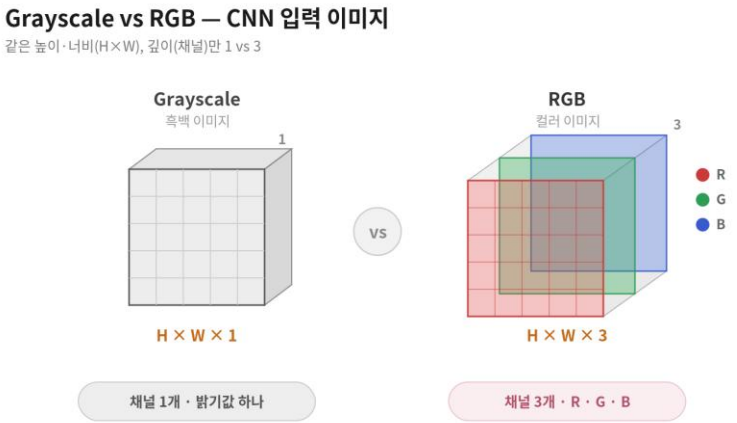
- Kernel은 3x3, 5x5 크기로 적용되는 것이 일반적
- Stride라는 지정된 간격으로 순차적 이동
- 각 kernel이 위치한 곳의 이미지와 kernel의 가중치를 곱한 후 더해 새로운 feature map 생성
- 지도 학습이므로 kernel 내의 가중치는 각 특징을 잘 찾아내도록 학습



RGB와 Gray scale

Gray scale에서는 깊이(채널)이 1인 Kernel을 사용함

하지만 RGB에서는 깊이가 3인 kernel 사용



Kernel이 여러 개

보통 한 합성곱층에 여러 개의 필터를 사용

필터 개수만큼의 feature map 생성

각 필터는 서로 다른 특징을 추출할 수 있음

Convolution Neural Network

Convolutional Layer

입력 데이터에서 특성을 추출하는 역할

Stride

- Kernel이 움직이는 보폭을 의미
- Stride가 커질수록 출력 크기는 작아짐

Padding

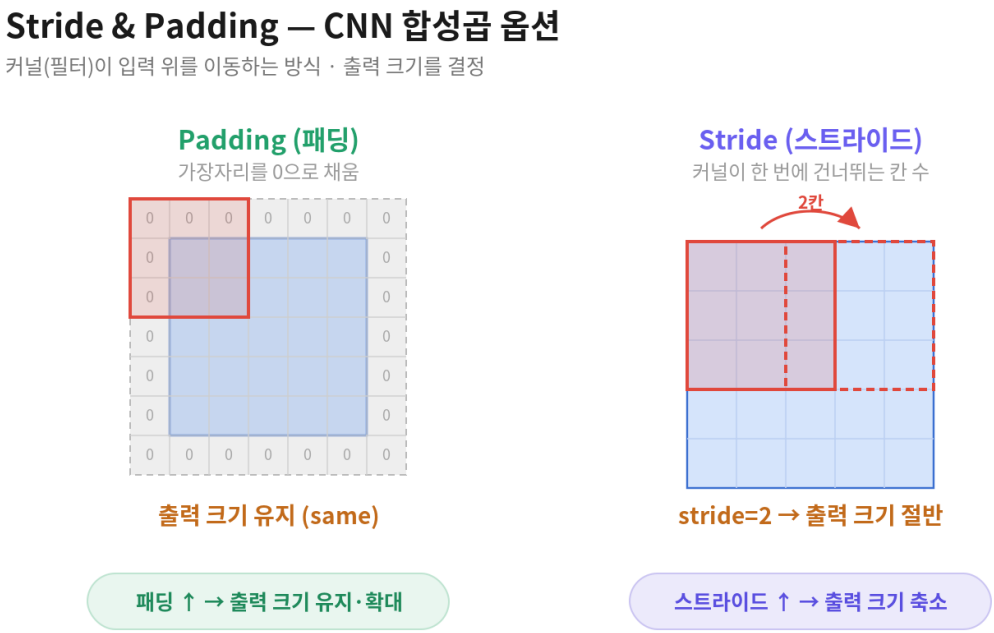
- 이미지의 가장자리에 0의 테두리를 두르는 것
- 합성곱을 하면 출력 크기가 작아짐
- 이미지의 가장자리는 필터가 지나가는 횟수가 부족하므로 정보가 적게 반영

출력 크기 계산 공식

- $O = \frac{I-F+2P}{S} + 1$
- O : 출력 크기
- I : 입력크기, F : 필터 크기, P : 패딩 크기, S : stride

ReLU 함수

- 합성곱 연산 후에는 보통 활성화 함수 적용
- ReLU 함수 이용하여 복잡한 비선형 패턴 표현



Pooling Layer

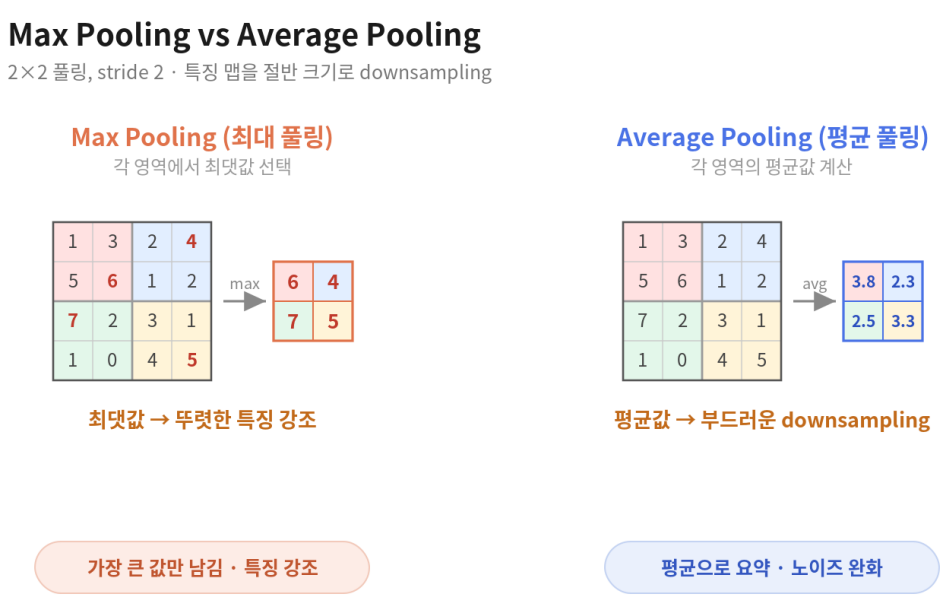
Feature map의 크기를 줄이는 과정

Max pooling

- 특정 영역에서 가장 큰 값만 남김

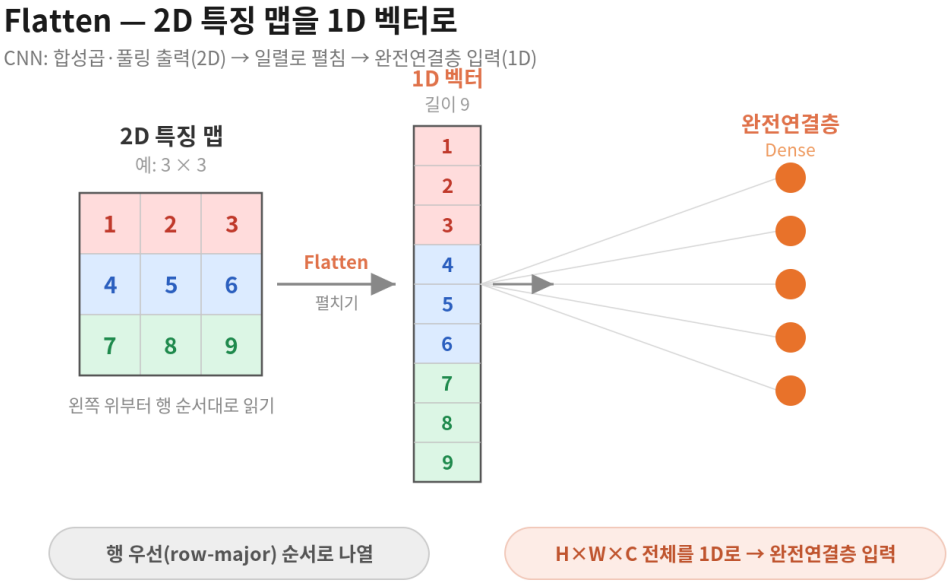
Average pooling

- 특정 영역의 평균값을 남김



Flatten

- 합성곱 층과 풀링 층을 지나며 2,3차원의 데이터형태 유지
- 분류를 위해 1차원 벡터로 펼침



Fully Connected Layer

일반적인 ANN층과 동일하며 Flatten된 벡터를 입력 받고 최종 분류를 수행함

Fully connected layer의 위치

- CNN의 뒷부분, 특징 추출이 끝난 뒤에 위치

Fully connected layer

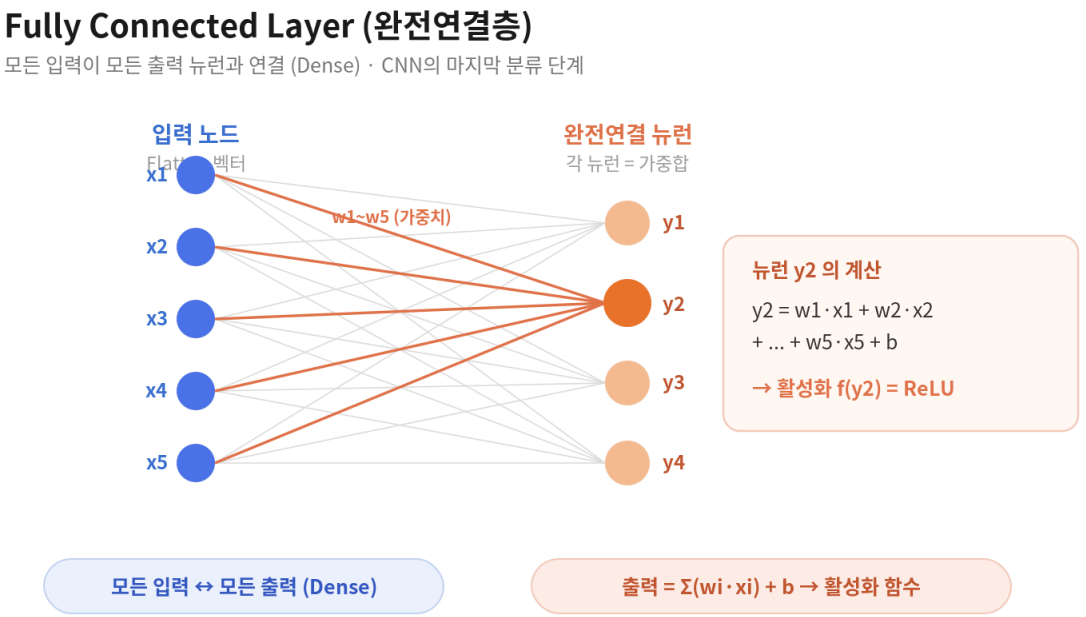
- 합성곱 층, 풀링 층을 지나며 추출된 특징을 이용해 최종 판단
- Flatten 한 후 Fully connected layer에서는 각 값을 node로 만들어 최종 분류 개수 만큼의 output node로 전달함

Fully connected layer의 계산

- ANN과 동일한 방식
- 입력 값마다 각 가중치가 곱해지고 bias를 더한 후 함수를 통과

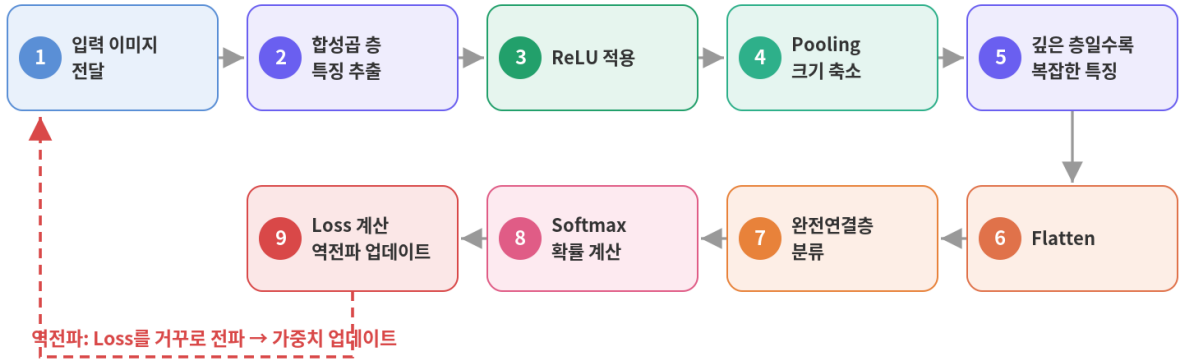
Softmax function

- Fully Connected Layer를 통과한 후 최종 출력은 아직 확률이 아닌 점수
- 따라서 softmax를 적용하여 확률로 변환



CNN 학습 과정과 특징

CNN의 학습 과정



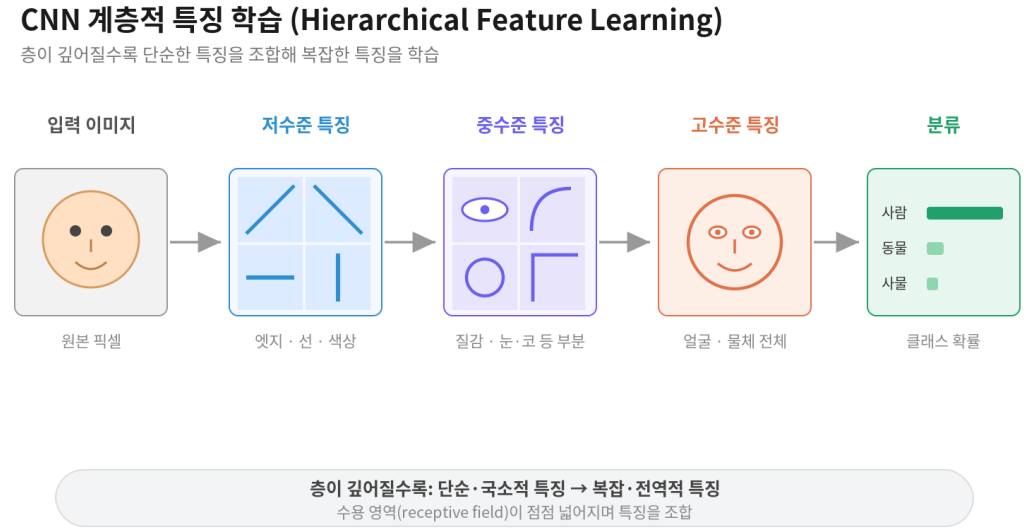
1~8단계: 순전파(Forward) · 9단계: 역전파(Backpropagation)로 학습 반복

CNN에서 학습되는 것

- 합성곱 층의 kernel의 가중치
- 완전연결층의 가중치

계층적 특징 학습

- 초기 층에서는 단순한 특징 학습
- 깊은 층으로 갈수록 더 복잡하고 추상적인 특징 학습



CNN 학습의 장점

가중치 공유

- CNN이 이미지 처리에서 효율적인 이유 중 하나
- 하나의 필터는 이미지 전체에서 같은 가중치 사용
- 같은 필터가 이미지 전체를 훑으며 같은 특징을 찾음
- 가중치 공유의 장점으로 파라미터 수의 감소, 효율적 계산 가능, 같은 특징이 어느 위치에 있어도 찾는 것이 있음

CNN의 주요 장점

특징	설명	핵심 이점
● 가중치 공유 Weight Sharing	하나의 필터(커널)를 이미지 전체 위치에 동일하게 적용	파라미터 수 급감 과적합 감소 · 학습 효율↑
● 지역적 연결 Local Connectivity	각 뉴런이 입력의 일부 영역(수용 영역)에만 연결	국소 패턴 포착 연산량 절감
● 위치 변화 강건성 Translation Robustness	풀링·가중치 공유로 물체 위치가 달라져도 같은 특징을 검출	이동에 강인 일반화 성능↑

지역적 연결

- ANN은 모든 입력이 모든 뉴런과 연결
- CNN에서는 필터가 이미지의 작은 영역만 봄
- 작은 영역에서 주변 픽셀의 관계를 효과적으로 관찰

위치 변화에 대한 강건성

- 이미지 속 물체가 조금 이동하더라도 어느정도 인식 가능
- 같은 필터를 이미지 전체에 적용하기 때문
- Pooling을 통해 위치 변화에 조금 둔감해질 수 있음

CNN 실전 코드

Colab MNIST 예제 검색하여, 코드 분석 진행

MNIST

- MNIST는 0~9의 흑백 숫자 이미지 데이터 (손글씨)

코드의 흐름

- MNIST 이미지에서 특징 추출
- 두개의 합성곱 층, 두개의 Pooling 층 존재
- Max Pooling 사용하여 이미지 크기를 절반으로 축소
- Flatten 후 완전 연결층으로 이동
- 숫자 0~9에 대한 점수 10개 출력

코드 분해 및 설명

```
import torch
import torchvision
import torch.nn as nn
```

- PyTorch 라이브러리 불러오기
- torchvision : PyTorch에서 이미지 관련 기능 제공하는 라이브러리
- 신경망을 만들기 위한 torch.nn 모듈을 불러옴

```
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
```

```
import numpy as np
import datetime
import os, sys
```

- DataLoader : 데이터셋에서 일정한 개수 씩 묶어 가져오는 기능
- Torchvision에서 제공하는 데이터 셋 관련 기능 가져옴(MNIST)
- transforms : 이미지 전처리 기능 가져오기

CNN 실전 코드

Colab MNIST 예제 검색하여, 코드 분석 진행

코드 분해 및 설명

```
from matplotlib.pyplot import imshow, imsave
%matplotlib inline
```

- matplotlib의 이미지 관련 함수 불러오기
- imshow : 이미지를 화면에 출력
- imsave : 이미지를 파일로 저장

```
MODEL_NAME = 'CNN'
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

- 문자열 CNN을 MODEL_NAME 변수에 저장
- CUDA를 사용할 수 있는 GPU 확인 코드
- 없으면 CPU 사용

```
class HelloCNN(nn.Module):
    """
        Simple CNN Clssifier
    """
    def __init__(self, num_classes=10):
        super(HelloCNN, self).__init__()
        self.conv = nn.Sequential(
            # (N, 1, 28, 28)
            nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            # (N, 32, 14, 14)
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            # (N, 64, 7, 7)
        )
        self.fc = nn.Sequential(
            nn.Linear(7*7*64, 512),
            nn.Dropout(p=0.5),
            nn.Linear(512, num_classes),
        )
```

- 새로운 CNN 모델 클래스 정의
- nn.Module을 상속받음
- 상속 받으면 PyTorch가 가중치나 편향, 역전파, 학습-평가 모드 등의 기능을 자동으로 관리가능

- __init__ : 모델 클래스가 생성되고, 이의 객체가 만들어질 때 자동으로 실행되는 생성자
- self : 현재 생성되고 있는 모델 자신
- num_classes=10 : 분류 클래스 10개로 설정
- super : 부모 클래스인 nn.model의 생성자를 실행, Pytorch가 내부적으로 모델의 레이어와 파라미터 정상등록 가능

CNN 실전 코드

Colab MNIST 예제 검색하여, 코드 분석 진행

코드 분해 및 설명

```
class HelloCNN(nn.Module):
    """
        Simple CNN Clssifier
    """
    def __init__(self, num_classes=10):
        super(HelloCNN, self).__init__()
        self.conv = nn.Sequential(
            # (N, 1, 28, 28)
            nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            # (N, 32, 14, 14)
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            # (N, 64, 7, 7)
        )
        self.fc = nn.Sequential(
            nn.Linear(7*7*64, 512),
            nn.Dropout(p=0.5),
            nn.Linear(512, num_classes),
        )
```

- self.conv = nn.Sequential(: 합성곱층, 활성화 함수, 풀링층을 순서대로 묶음
- 이 코드에서는 Conv2d는 ReLU -> Maxpool2d -> Conv2d -> ReLU -> MaxPool2d의 순서로 구성
- self.conv는 self의 이 변수가 모델 객체에 소속된다는 뜻
- CNN의 입력 Tensor 크기는 다음과 같다
- N : 배치에 포함된 이미지 수
- 1 : 이미지 채널 수
- 28 : 이미지 높이
- 28 : 이미지 너비

- nn.Conv2d(1,32,kernel_size=3,stride=1,padding=1
- 합성곱의 하이퍼 파라미터인 stride와 padding, 필터 개수=32, kernel 크기를 설정, 채널은 1
- 필터 한 개의 크기는 (1,3,3) 이고 32개의 필터이므로 전체 가중치의 크기는 (32,1,3,3) 임
- $H_{out} = \left[\frac{H_{in}+2P-K}{S}\right] + 1$
- H는 크기, P는 패딩, K는 kernel size, S는 stride
- MaxPooling 사용, 2x2영역에서 가장 큰 값 선택
- 풀링층의 strid는 2, 높이와 너비가 절반으로 감소

CNN 실전 코드

Colab MNIST 예제 검색하여, 코드 분석 진행

코드 분해 및 설명

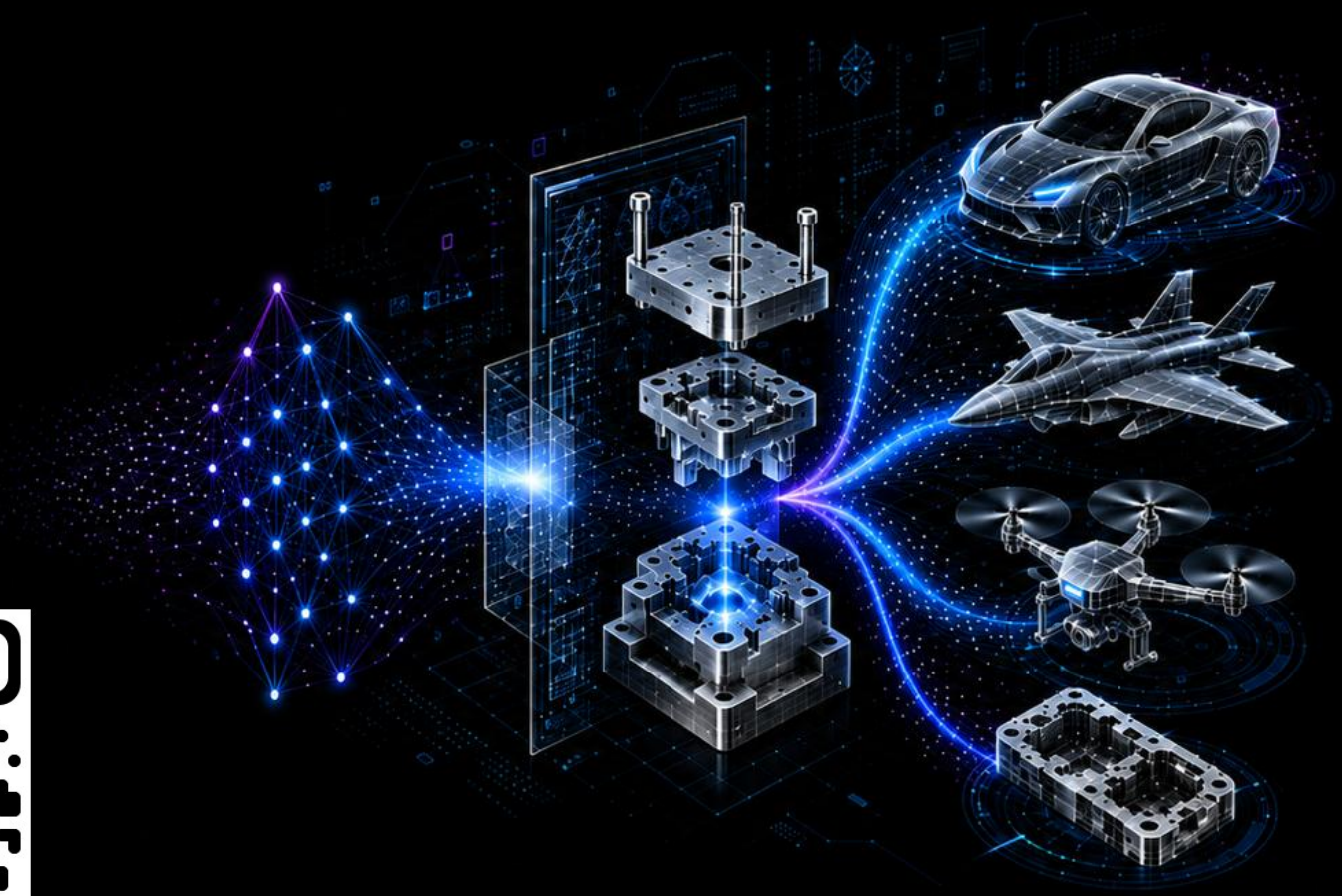
```
class HelloCNN(nn.Module):
    """
        Simple CNN Clssifier
    """
    def __init__(self, num_classes=10):
        super(HelloCNN, self).__init__()
        self.conv = nn.Sequential(
            # (N, 1, 28, 28)
            nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            # (N, 32, 14, 14)
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            # (N, 64, 7, 7)
        )
        self.fc = nn.Sequential(
            nn.Linear(7*7*64, 512),
            nn.Dropout(p=0.5),
            nn.Linear(512, num_classes),
        )
```

- 두 번째 합성곱 층에서 입력 채널 32개
- 출력 feature map 64개
- 두 번째 conv층의 필터 깊이도 32
- Maxpooling 지나면서 크기 절반으로 감소
- self.fc=nn.Sequential(: 완전 연결층
- 입력 3136개 -> 출력 512개 (활성화함수 없음)
- Drop out 0.5 (테스트에서만)

```
def forward(self, x):
    y_ = self.conv(x) # (N, 64, 7, 7)
    y_ = y_.view(y_.size(0), -1) # (N, 64*7*7)
    y_ = self.fc(y_)
    return y_
```

- 순전파 함수
- 입력 x에 대해 어떤 순서로 계산할지 정의
- y_=self.conv(x)로 앞서 정의한 합성곱 블록에 통과시킴
- y_=y_.view는 3차원 feature map을 1차원 vector로 펼침
- 이후 완전연결층 지남

Any Questions?



이 름 양 선 웅

소 속 한양대학교 기계공학과

이메일 sunwoongy@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab